

Manual de instalación de SomnoAI

Este manual explica cómo poner SomnoAI en marcha de dos formas: en una máquina local con Docker Compose, y en la nube con AWS ECS Fargate.

1. Qué se instala

SomnoAI son dos contenedores que se ejecutan juntos. El primero es la API, hecha con FastAPI, que analiza los registros EDF, estima la edad cerebral y el BAI, y responde en el puerto 8000. El segundo es el tablero web, servido con nginx en el puerto 8050.

El tablero es JavaScript que corre en el navegador del usuario y llama a la API directamente, armando la dirección con el nombre del host de la página más el puerto 8000. Por eso los dos puertos tienen que estar abiertos, y los dos contenedores tienen que compartir la misma dirección. En ECS con Fargate esto se cumple solo, porque los contenedores de una misma tarea comparten una IP.

Los dos Dockerfile se construyen desde la raíz del repositorio y no desde la carpeta donde están. La API lo necesita porque su archivo de dependencias instala el modelo entrenado desde la carpeta model-pkg. De ahí que los comandos de más abajo lleven la opción -f.

2. Requisitos

- Git.
- Docker y Docker Compose, si se va a instalar en local. Se descargan de <https://docs.docker.com/desktop/>
- Una cuenta de AWS con acceso a EC2, ECR y ECS, si se va a desplegar en la nube.
- Unos 5 GB de disco libre. La imagen de la API pesa alrededor de 1,15 GB y los datos de ejemplo unos 148 MB.

3. Instalación local con Docker Compose

Clone el repositorio:

```
git clone https://github.com/andriunet/SomnoAI.git
cd SomnoAI
```

Descargue los datos de ejemplo. Son tres polisomnografías reales de la base Sleep-EDFx de PhysioNet. Sin ellas la API arranca igual, pero el tablero aparece sin registros.

```
sh backend/data/demo/download_demos.sh
```

Levante los contenedores:

```
docker compose up --build
```

El primer arranque tarda cerca de medio minuto más de lo normal, porque la API analiza los tres ejemplos antes de empezar a responder.

Cuando termine, el tablero queda en <http://localhost:8050> y la API en <http://localhost:8000>, con su documentación interactiva en <http://localhost:8000/docs>. El usuario es superusuario y la contraseña somnoai2026.

Para detener todo:

```
docker compose down
```

4. Despliegue en AWS ECS Fargate

El camino es el siguiente: se levanta una instancia EC2 pequeña que sirve solo para construir las dos imágenes, las imágenes se suben al registro ECR, y ECS con Fargate las ejecuta. La instancia puede apagarse en cuanto termina la subida.

4.1 Credenciales

Si usa AWS Academy, inicie el Laboratorio de aprendizaje y abra, en el menú de la esquina superior derecha, la opción AWS Details y dentro de ella AWS CLI. Copie el `aws_access_key_id`, el `aws_secret_access_key` y el `aws_session_token`, y guárdelos en un archivo de texto.

Estas credenciales son temporales y caducan al cerrar la sesión del laboratorio. Si más adelante un comando falla con el error `ExpiredToken`, vuelva a copiarlas.

4.2 Instancia de construcción

En la consola de AWS entre a EC2 y lance una instancia con estos valores:

- Nombre: somnoai-build
- Imagen: Ubuntu Server 24.04 LTS
- Tipo: t3.micro
- Par de claves: cree uno nuevo y descargue el archivo `.pem`
- Almacenamiento: 20 GB
- Grupo de seguridad: permita la entrada por los puertos 22, 8000 y 8050

Conéctese por SSH:

```
chmod 400 llave.pem
ssh -i llave.pem ubuntu@IP
```

4.3 Preparación de la máquina

La t3.micro tiene alrededor de 1 GB de memoria, que es poco para construir imágenes grandes. Conviene activar un archivo de intercambio antes de empezar:

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
```

Actualice el índice de paquetes e instale lo necesario:

```
sudo apt-get update
sudo apt-get install -y zip unzip ca-certificates curl gnupg
```

4.4 AWS CLI

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
aws --version
```

Configure las credenciales con el comando `aws configure`, indicando `us-east-1` como región. Si son credenciales de AWS Academy hace falta además el token de sesión, que `aws configure` no pide; lo más simple es pegar el bloque completo en el archivo de credenciales:

```
mkdir -p ~/.aws
nano ~/.aws/credentials
printf '[default]\nregion = us-east-1\noutput = json\n' > ~/.aws/config
```

Compruebe que quedó bien:

```
aws sts get-caller-identity
```

4.5 Docker

Elimine versiones anteriores. Si no hay ninguna el comando da error y puede continuar sin problema:

```
sudo apt-get remove docker docker-engine docker.io containerd runc
```

Agregue la llave y el repositorio de Docker:

```

sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu $(.
/etc/os-release && echo $VERSION_CODENAME) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null

```

Instale Docker y verifique:

```

sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
sudo docker run hello-world

```

4.6 Repositorios en ECR

En la consola de AWS busque el servicio ECR y, en el registro privado, cree dos repositorios con los valores por defecto: uno llamado somnoai-api y otro somnoai-front.

Copie las URI de los dos. Tienen esta forma:

```
123456789012.dkr.ecr.us-east-1.amazonaws.com/somnoai-api
```

También se pueden crear desde la línea de comandos:

```

aws ecr create-repository --repository-name somnoai-api --region us-east-1
aws ecr create-repository --repository-name somnoai-front --region us-east-1

```

4.7 Construcción de las imágenes

Clone el repositorio en la instancia:

```

git clone https://github.com/andriunet/SomnoAI.git
cd SomnoAI

```

Construya la imagen de la API. El argumento BAKE_DEMOS incluye los datos de ejemplo dentro de la imagen, cosa que en Fargate es necesaria porque allí no se pueden montar carpetas de la máquina anfitriona como sí hace Docker Compose en local. Sin ese argumento la aplicación funciona, pero el tablero arranca sin registros.

```

sudo docker build -f backend/Dockerfile --build-arg BAKE_DEMOS=1 -t somnoai-
api:latest .

```

Construya la imagen del tablero, también desde la raíz:

```

sudo docker build -f frontend/Dockerfile -t somnoai-front:latest .

```

Revise que las dos quedaron creadas:

```
sudo docker images
```

4.8 Subida de las imágenes a ECR

Reemplace repoURI por la URI del repositorio correspondiente en cada comando:

```
aws ecr get-login-password --region us-east-1 | sudo docker login --username AWS --password-stdin repoURI
```

```
sudo docker tag somnoai-api:latest repoURI
sudo docker push repoURI
```

Repita el etiquetado y la subida con la imagen del tablero. La subida de la API tarda varios minutos, porque son cerca de 1,15 GB.

4.9 Clúster

En la consola busque el servicio ECS, entre a Clústeres y cree uno llamado somnoai-cluster, seleccionando AWS Fargate como infraestructura. Si al crearlo aparece un mensaje de error puede ignorarlo.

4.10 Definición de tarea

En el menú izquierdo entre a Definiciones de tareas y cree una nueva con estos valores:

- Nombre de familia: somnoai-task
- Tipo de lanzamiento: Fargate
- CPU y memoria: 1 vCPU y 3 GB
- Rol de tarea y rol de ejecución de tareas: LabRole

Agregue dos contenedores, los dos marcados como esenciales. El primero se llama api, usa la URI de la imagen somnoai-api y mapea el puerto 8000. El segundo se llama front, usa la imagen somnoai-front y mapea el puerto 8050.

El límite de 3 GB de memoria no es opcional. El análisis de un registro completo es la operación que más memoria consume, y con menos la tarea se reinicia mientras precomputa los ejemplos.

Si define una comprobación de estado para el contenedor de la API, déle un período de inicio de 300 segundos. El primer arranque analiza los tres ejemplos antes de responder, y con un período más corto ECS da la tarea por muerta.

En la carpeta manuales del repositorio está el archivo task-definition.json, que trae esta misma definición para registrarla desde la línea de comandos:

```
aws ecs register-task-definition --cli-input-json file://manuales/task-  
definition.json
```

4.11 Servicio

Vuelva al clúster, entre a Servicios y cree uno nuevo. Seleccione la definición de tarea recién creada con su revisión más reciente, póngale un nombre, deje una tarea deseada y asegúrese de que la red use una subred pública con IP pública habilitada.

4.12 Puertos

En la pestaña Redes del servicio abra el grupo de seguridad, edite las reglas de entrada y habilite el tráfico por los puertos 8000 y 8050 desde cualquier dirección IPv4.

Abrir solo el 8050 no basta. Como el tablero llama a la API desde el navegador del usuario, si el 8000 está cerrado la página carga pero queda sin datos.

4.13 Comprobación

Espere dos o tres minutos a que la tarea pase al estado RUNNING, ábrala y copie su IP pública. Con ella:

- El tablero queda en IP:8050
- La documentación de la API en IP:8000/docs
- La comprobación de estado en IP:8000/api/v1/health

La comprobación de estado debe responder algo parecido a esto:

```
{"status":"ok","records":3,"model":"edad-cerebral-0.3.0"}
```

Los tres registros son los ejemplos precomputados. Si aparece un cero, la imagen se construyó sin el argumento BAKE_DEMOS.

Entre al tablero con el usuario superusuario y la contraseña somnoai2026.

5. Publicar una versión nueva

Después de cambiar el código, reconstruya la imagen en la instancia y vuelva a subirla:

```
cd ~/SomnoAI  
git pull  
sudo docker build -f backend/Dockerfile --build-arg BAKE_DEMOS=1 -t somnoai-  
api:latest .  
sudo docker tag somnoai-api:latest repoURI  
sudo docker push repoURI
```

En la consola de ECS, entre a la definición de tarea y cree una revisión nueva apuntando a la imagen más reciente. Después actualice el servicio para que use esa revisión. ECS reemplaza la tarea que está corriendo.

Desde la línea de comandos el último paso es:

```
aws ecs update-service --cluster somnoai-cluster --service somnoai-service --force-new-deployment
```

6. Problemas frecuentes

El tablero carga pero no muestra datos. Falta abrir el puerto 8000 en el grupo de seguridad de la tarea.

La comprobación de estado responde con cero registros. La imagen se construyó sin el argumento `BAKE_DEMOS`. Reconstrúyala y vuelva a subirla.

La tarea se reinicia una y otra vez. Le falta memoria. Súbala a 3 GB en la definición de tarea.

La tarea muere antes de llegar a responder. La comprobación de estado venció mientras la aplicación todavía analizaba los ejemplos. Suba el período de inicio a 300 segundos.

La construcción de la imagen falla por falta de memoria. La `t3.micro` tiene alrededor de 1 GB. Active el archivo de intercambio de la sección 4.3.

Los comandos de `aws` empiezan a fallar con `ExpiredToken`. Las credenciales del laboratorio caducaron. Cópielas de nuevo.

La subida a ECR responde que el acceso fue denegado. La sesión del registro caducó. Repita el comando de login de la sección 4.8.

El histórico aparece vacío después de reiniciar la tarea. Es lo esperado. El almacenamiento de la tarea es temporal, así que la base de datos se pierde y los ejemplos se vuelven a calcular.

Para ver los mensajes de la aplicación:

```
aws logs tail /ecs/somnoai --follow --region us-east-1
```

7. Desmontar el despliegue

Para dejar de consumir recursos:

```
aws ecs update-service --cluster somnoai-cluster --service somnoai-service --
desired-count 0
aws ecs delete-service --cluster somnoai-cluster --service somnoai-service --force
aws ecs delete-cluster --cluster somnoai-cluster
aws ecr delete-repository --repository-name somnoai-api --force
aws ecr delete-repository --repository-name somnoai-front --force
aws ec2 terminate-instances --instance-ids ID_DE_LA_INSTANCIA
```

8. Advertencias

El usuario y la contraseña del prototipo están escritos en el archivo backend/app/config.py. Si el despliegue tiene IP pública, cualquiera que la conozca puede entrar. Conviene cambiarlos antes de usar el sistema para algo que no sea una demostración.

El despliegue funciona sobre HTTP, sin cifrado, de modo que las credenciales viajan en claro. Para un uso real habría que poner delante un balanceador con certificado.

El almacenamiento es temporal. La base de datos y las señales derivadas viven dentro del contenedor y se pierden en cada reinicio. Para conservarlas haría falta un sistema de archivos externo o una base de datos aparte.

SomnoAI es un prototipo académico. No es un dispositivo médico y no debe usarse para tomar decisiones clínicas.